1. Data Cleaning & Preparation

```
# Looking for duplicate Order_ID records
SELECT
    Order_ID, COUNT(Order_ID) AS dup_ordrs
FROM
    orders
GROUP BY Order_ID
HAVING dup_ordrs > 1;
# When executed, the query shows that there are no duplicate entries pertaining to Order_ID
```

Order_ID	dup_ordrs

```
# The following query is implemented for replacing null values in Traffic_Delay_Min with the average delay for that route using a window function
SELECT
    Route_ID,
    Start_Location,
    End_Location,
    Distance_KM,
    Average_Travel_Time_Min,
    round(COALESCE(Traffic_Delay_Min, avg(Traffic_Delay_Min) OVER (PARTITION BY Route_ID))) AS Traffic_Delay_Min
FROM
    routes;
```

Route_ID	Start_Location	End_Location	Distance_KM	Average_Travel_Time_Min	Traffic_Delay_Min
R001	New Aimeechester	Sabrinaland	130.1	212	36
R002	South Brianchester	Huffmanmouth	191.9	89	22
R003	East Amy	South Katherinemouth	110.5	190	11
R004	Ramosside	Stevenview	136.9	32	3
R005	New Jessica	Davismouth	71.4	191	26
R006	Lake Robinhaven	East Christopherville	29.4	183	42
R007	Aaronhaven	Jeanneberg	74.3	263	51
R008	South Davidborough	Carsonberg	52.4	215	34
R009	Bellfurt	Lake Marvin	126.4	209	46
R010	North Aliceside	Lake Jennifer	115	172	40
R011	Lowemouth	Samanthaborough	170.1	178	25
R012	West Calvinview	Port Audreyberg	99.4	110	29
R013	South Johnville	Port Scott	37.3	40	38
R014	North Edward	Matthewport	125.3	55	58
R015	South Jenniferfurt	Lindsayton	34.6	115	29
R016	Lake Kathrynchester	Port Michael	25.1	166	8
R017	Port Tammybury	Abbottside	24.6	75	8
R018	Lake Jessicastad	Samuelport	142.2	265	11
R019	Lake Oscarfurt	New Ivanshire	184.7	49	37
R020	Juliestad	South Jonathanshire	94.1	171	23

```
# The following queries use DDL commands to change all date columns into YYYY-MM-DD format
ALTER TABLE orders MODIFY COLUMN Order_Date DATE, MODIFY COLUMN Expected_Delivery_Date DATE,
MODIFY COLUMN Actual_Delivery_Date DATE;
SELECT
    *
FROM
    orders;
```

Order_ID	Customer_ID	Warehouse_ID	Route_ID	Order_Date	Expected_Delivery_Date	Actual_Delivery_Date	Delivery_Status
O0001	C0080	W004	R005	08-06-2025	11-06-2025	11-06-2025	On Time
O0002	C0566	W009	R004	22-06-2025	27-06-2025	29-06-2025	Delayed
O0003	C0899	W008	R013	04-06-2025	09-06-2025	11-06-2025	Delayed

```
# and another query for shipment_tracking
ALTER TABLE shipment_tracking MODIFY COLUMN Checkpoint_Time DATE;
SELECT
    *
FROM
    shipment_tracking;
```

Shipment_ID	Order_ID	Checkpoint	Checkpoint_Time	Delay_Reason
SO00011	O0001	Checkpoint 1	09-06-2025	Weather
SO00012	O0001	Checkpoint 2	09-06-2025	Sorting Delay
SO00021	O0002	Checkpoint 1	22-06-2025	Sorting Delay

Note : I have only used a preview of 3 rows in the result tables on this on this page as there are about 300 records and importing all of them will make the slide look messy. For further clarity you can access the SQL script (Data_Cleaning_&_Preparation) which is attached with this PPT.

The following query is used to make sure that no Actual_Delivery_Date is before Order_Date and also implements a case clause to check for conditions and flagging the the date check accordingly

```
SELECT
    Order_ID,
    Order_Date,
    Actual_Delivery_Date,
    CASE
        WHEN Actual_Delivery_Date < Order_Date THEN 'Invalid'
        ELSE 'Valid'
    END AS Delivery_dt_chk
FROM
    orders;
```

Order_ID	Order_Date	Actual_Delivery_Date	Delivery_dt_chk
O0001	08-06-2025	11-06-2025	Valid
O0002	22-06-2025	29-06-2025	Valid
O0003	04-06-2025	11-06-2025	Valid
O0004	28-06-2025	07-07-2025	Valid
O0005	04-06-2025	10-06-2025	Valid

Note : I have only used a preview of 5 rows in the result table above as there are 300 records here and many of the following result tables as well. After observing the result set in this case, it's prominent that all the orders are valid i.e., no actual delivery date is before date when the order was placed.

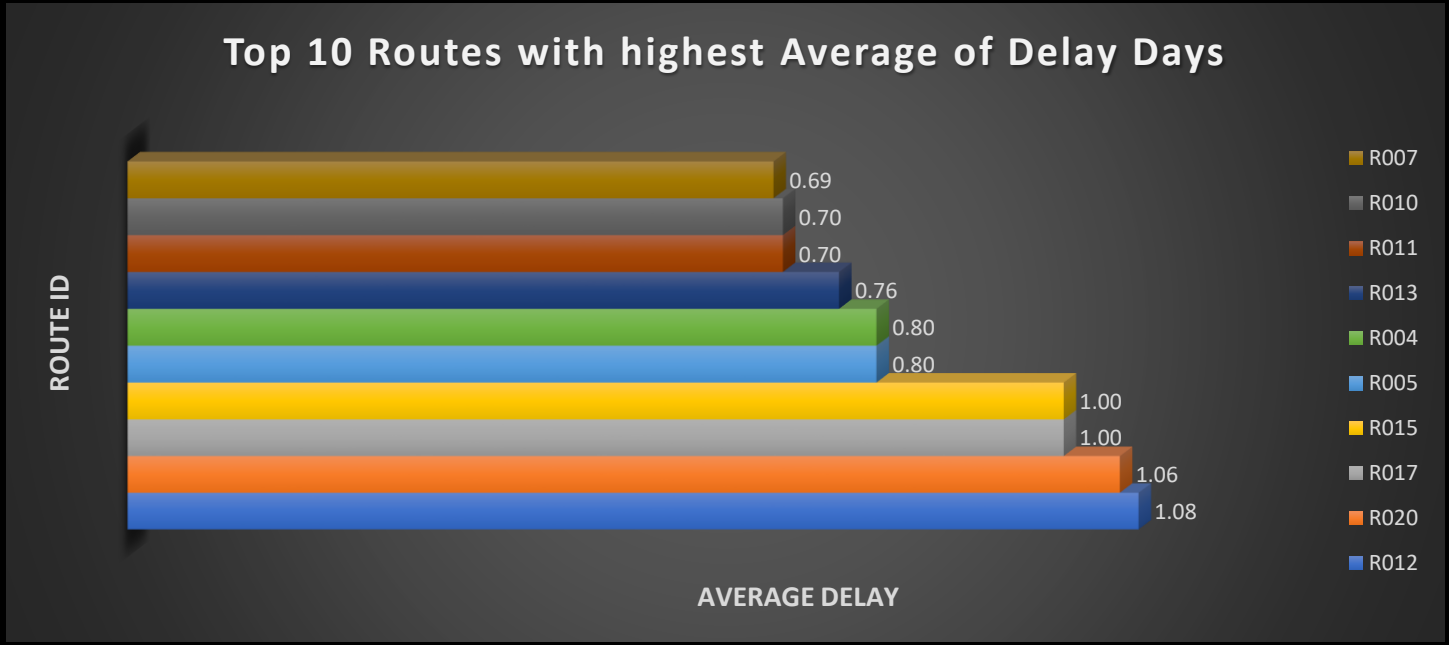
2. Delivery Delay Analysis

```
# The following query calculates the delivery delay in days for each order
SELECT
    Order_ID,
    Order_Date,
    Expected_Delivery_Date,
    Actual_Delivery_Date,
    DATEDIFF(Actual_Delivery_Date,
            Expected_Delivery_Date) AS Delivery_Delay_Days
FROM
    orders;
```

Order_ID	Order_Date	Expected_Delivery_Date	Actual_Delivery_Date	Delivery_Delay_Days
O0001	08-06-2025	11-06-2025	11-06-2025	0
O0002	22-06-2025	27-06-2025	29-06-2025	2
O0003	04-06-2025	09-06-2025	11-06-2025	2
O0004	28-06-2025	05-07-2025	07-07-2025	2
O0005	04-06-2025	09-06-2025	10-06-2025	1
O0006	17-07-2025	19-07-2025	19-07-2025	0
O0007	14-06-2025	17-06-2025	18-06-2025	1
O0008	23-06-2025	30-06-2025	02-07-2025	2

```
# The query used below identifies and presents the top 10 delayed routes based on average delay days
SELECT
    Route_ID,
    ROUND(AVG(DATEDIFF(Actual_Delivery_Date,
                       Expected_Delivery_Date)),
            2) AS avg_del
FROM
    orders
GROUP BY Route_ID
ORDER BY avg_del DESC
LIMIT 10;
```

Route_ID	avg_del
R012	1.08
R020	1.06
R017	1.00
R015	1.00
R005	0.80
R004	0.80
R013	0.76
R011	0.70
R010	0.70
R007	0.69



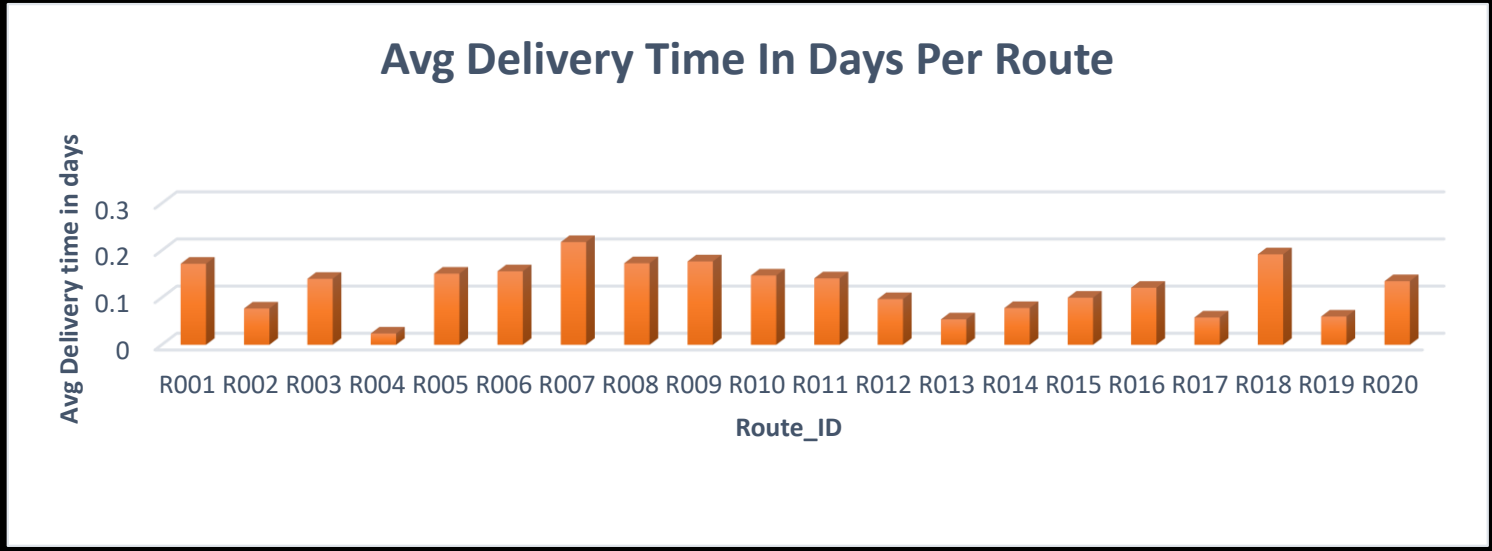
```
# The following query uses dense_rank() function as a window function to assign ranks to all
orders by delay within each warehouse
Select Order_ID, Warehouse_ID,
Expected_Delivery_Date,
Actual_Delivery_Date,
datediff(Actual_Delivery_Date, Expected_Delivery_Date) as Delay,
dense_rank() over (partition by Warehouse_ID order by datediff(Actual_Delivery_Date,
Expected_Delivery_Date)
Desc)
 as del_rank
 from orders;
```

Order_ID	Warehouse_ID	Expected_Delivery_Date	Actual_Delivery_Date	Delay	del_rank
O0276	W001	20-06-2025	22-06-2025	2	1
O0275	W001	12-07-2025	14-07-2025	2	1
O0004	W001	05-07-2025	07-07-2025	2	1
O0222	W001	30-06-2025	02-07-2025	2	1
O0065	W001	17-07-2025	19-07-2025	2	1
O0274	W001	22-07-2025	24-07-2025	2	1
O0263	W001	03-06-2025	05-06-2025	2	1
O0238	W001	02-07-2025	04-07-2025	2	1
O0220	W001	26-06-2025	28-06-2025	2	1
O0160	W001	01-07-2025	02-07-2025	1	2
O0223	W001	08-07-2025	09-07-2025	1	2
O0007	W001	17-06-2025	18-06-2025	1	2
O0186	W001	24-07-2025	25-07-2025	1	2
O0057	W001	15-06-2025	16-06-2025	1	2

3. Route Optimization Insights

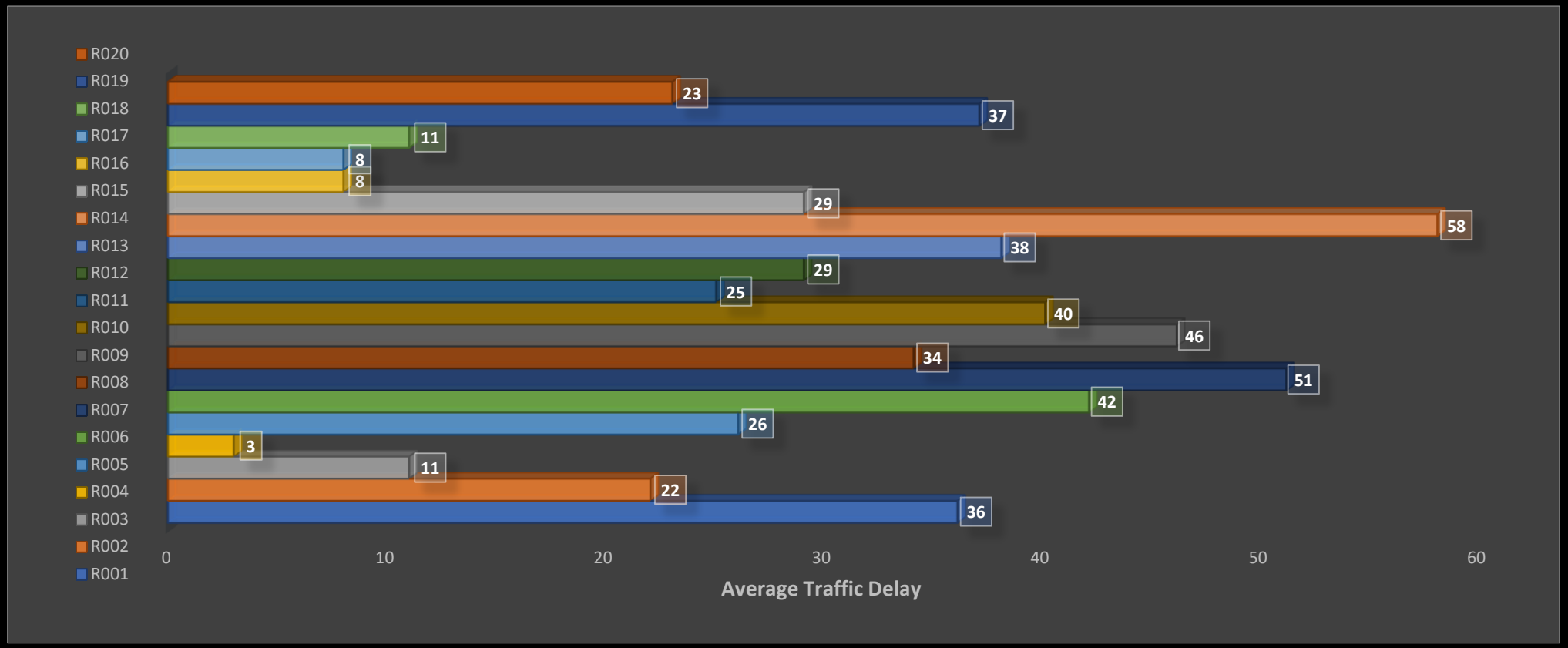
```
# The following query calculates Average delivery time in days for each route
SELECT
    Route_ID,
    Start_Location,
    End_Location,
    ROUND((Average_Travel_Time_Min + Traffic_Delay_Min) / 1440,
            3) AS Avg_Delivery_Time_Days
FROM
    routes;
```

Route_ID	Start_Location	End_Location	Avg_Delivery_Time_Days
R001	New Aimeechester	Sabrinaland	0.172
R002	South Brianchester	Huffmanmouth	0.077
R003	East Amy	South Katherinemouth	0.14
R004	Ramosside	Stevenview	0.024
R005	New Jessica	Davismouth	0.151
R006	Lake Robinhaven	East Christopherville	0.156
R007	Aaronhaven	Jeanneberg	0.218
R008	South Davidborough	Carsonberg	0.173
R009	Bellfurt	Lake Marvin	0.177
R010	North Aliceside	Lake Jennifer	0.147
R011	Lowemouth	Samanthaborough	0.141
R012	West Calvinview	Port Audreyberg	0.097
R013	South Johnville	Port Scott	0.054
R014	North Edward	Matthewport	0.078
R015	South Jenniferfurt	Lindsayton	0.1
R016	Lake Kathrynchester	Port Michael	0.121
R017	Port Tammybury	Abbottside	0.058
R018	Lake Jessicastad	Samuelport	0.192
R019	Lake Oscarfurt	New Ivanshire	0.06
R020	Juliestad	South Jonathanshire	0.135



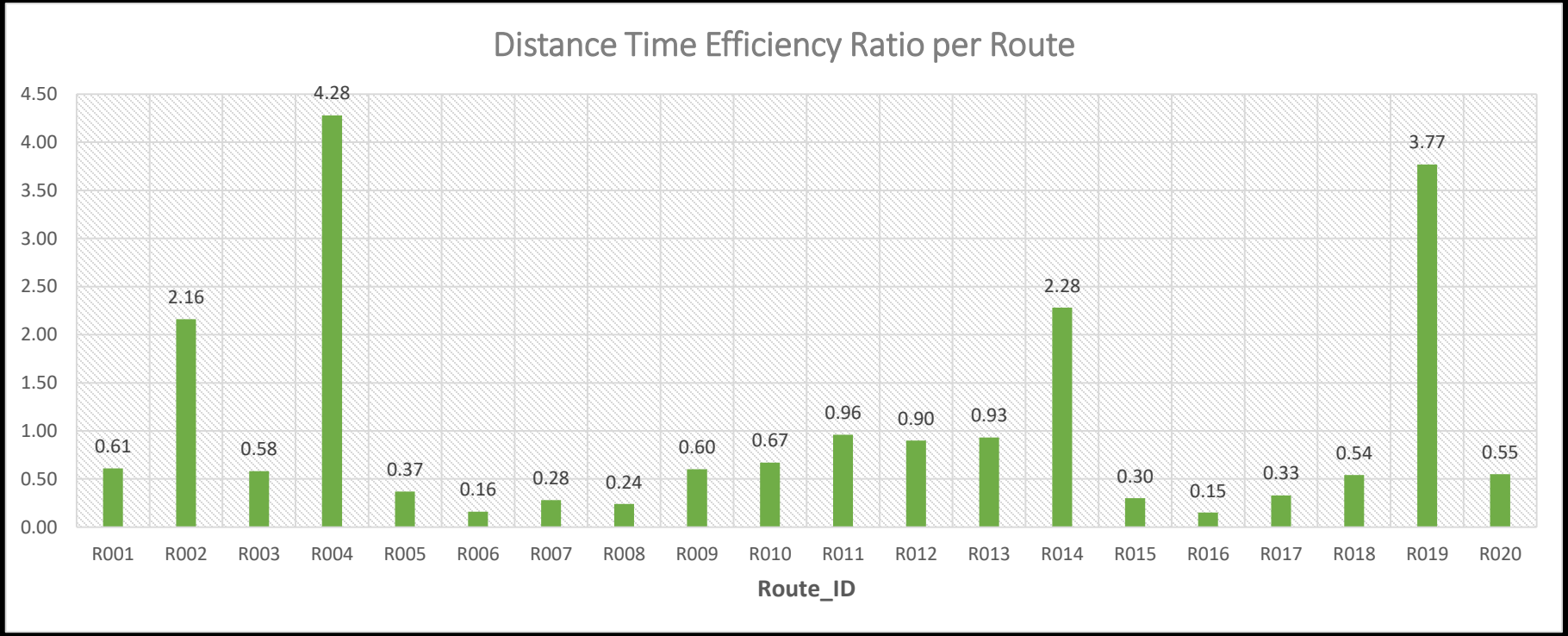
```
# The query below calculates Average traffic delay for each route
SELECT
    Route_ID, ROUND(AVG(Traffic_Delay_Min)) AS avg_trf_del
FROM
    routes
GROUP BY Route_ID;
```

Route_ID	avg_trf_del
R001	36
R002	22
R003	11
R004	3
R005	26
R006	42
R007	51
R008	34
R009	46
R010	40
R011	25
R012	29
R013	38
R014	58
R015	29
R016	8
R017	8
R018	11
R019	37
R020	23



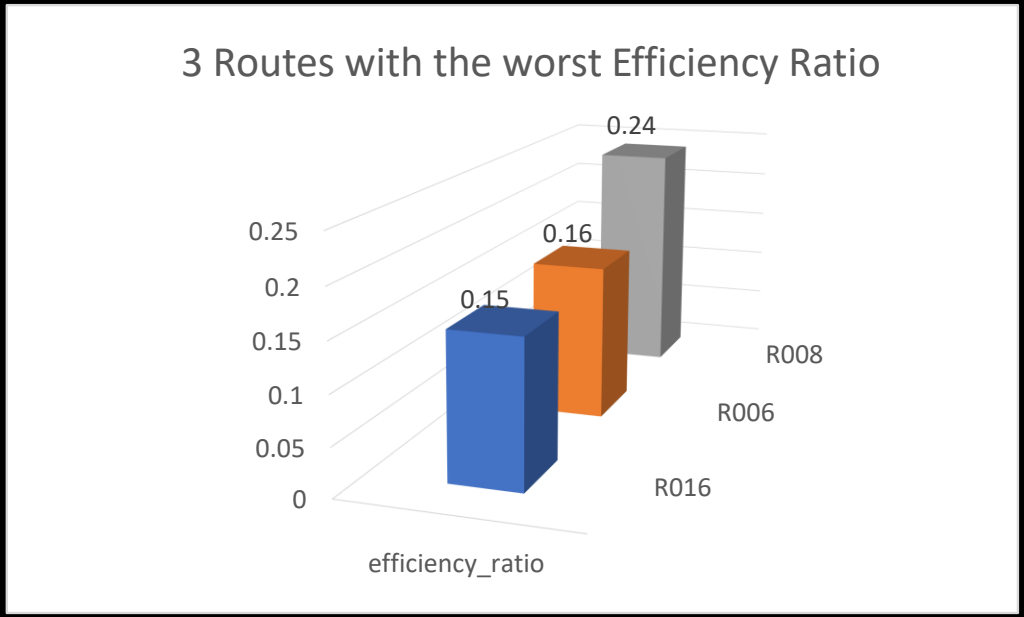
```
# The following query is used to calculate Distance-to-time efficiency ratio using the given formula
(Distance_KM / Average_Travel_Time_Min) for each route
SELECT
    Route_ID,
    ROUND(Distance_KM / Average_Travel_Time_Min, 2) AS dist_time_efficiency
FROM
    routes;
```

Route_ID	dist_time_efficiency
R001	0.61
R002	2.16
R003	0.58
R004	4.28
R005	0.37
R006	0.16
R007	0.28
R008	0.24
R009	0.6
R010	0.67
R011	0.96
R012	0.9
R013	0.93
R014	2.28
R015	0.3
R016	0.15
R017	0.33
R018	0.54
R019	3.77
R020	0.55



```
# The query mentioned below identifies and shows bottom 3 routes with the worst efficiency ratio
SELECT
    Route_ID,
    Distance_KM,
    Average_Travel_Time_Min,
    ROUND(Distance_KM / IFNULL(Average_Travel_Time_Min, 0),
            2) AS efficiency_ratio
FROM
    routes
ORDER BY efficiency_ratio ASC
LIMIT 3;
```

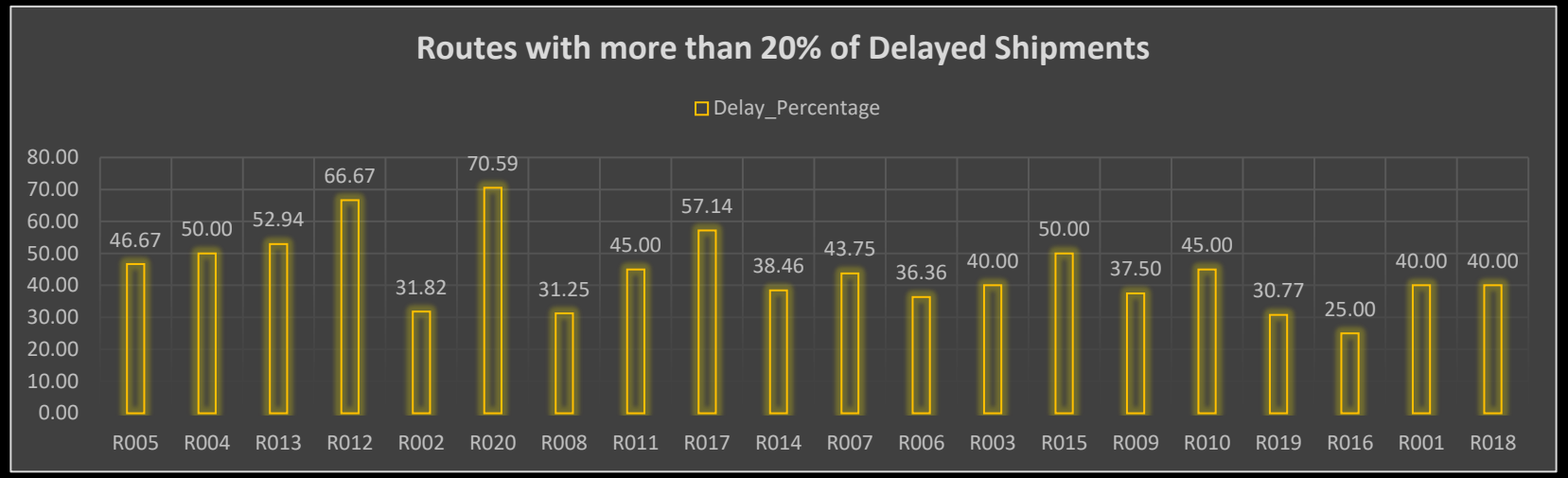
Route_ID	Distance_KM	Average_Travel_Time_Min	efficiency_ratio
R016	25.1	166	0.15
R006	29.4	183	0.16
R008	52.4	215	0.24



```
# The following query identifies and gives routes with >20% delayed shipments without using joins
as there is already a recognizable relationship between orders, routes and shipmentment_tracking
tables (Route_ID in routes table to Route_ID in orders tables and Order_ID in orders table to
Order_ID in shipment_tracking table)
SELECT
    o.Route_ID,
    COUNT(o.Order_ID) AS Total_Shipments,
    SUM(CASE
        WHEN o.Delivery_Status = 'Delayed' THEN 1
        ELSE 0
    END) AS Delayed_Count,
    (SUM(CASE
        WHEN o.Delivery_Status = 'Delayed' THEN 1
        ELSE 0
    END) / COUNT(o.Order_ID)) * 100 AS Delay_Percentage
FROM
    Orders o
GROUP BY o.Route_ID
HAVING Delay_Percentage > 20;
```

Note : The result table of the query mentioned above is in the next slide.

Route_ID	Delay_Percentage
R005	46.67
R004	50.00
R013	52.94
R012	66.67
R002	31.82
R020	70.59
R008	31.25
R011	45.00
R017	57.14
R014	38.46
R007	43.75
R006	36.36
R003	40.00
R015	50.00
R009	37.50
R010	45.00
R019	30.77
R016	25.00
R001	40.00
R018	40.00



 # Some recommendations of potential routes for optimization are :

-- 1. Routes with a high rate of delay like Route_ID - R002 are at critical risk and need immediate action. In this case, the route might not be the issue, maybe it's the departure time which can be affected by traffic or possible warehouse bottlenecks. UPS can perform Time of the Day Analysis to get a better insight on the matter and also switch departured timings by a few hours to avoid the identified congestion.

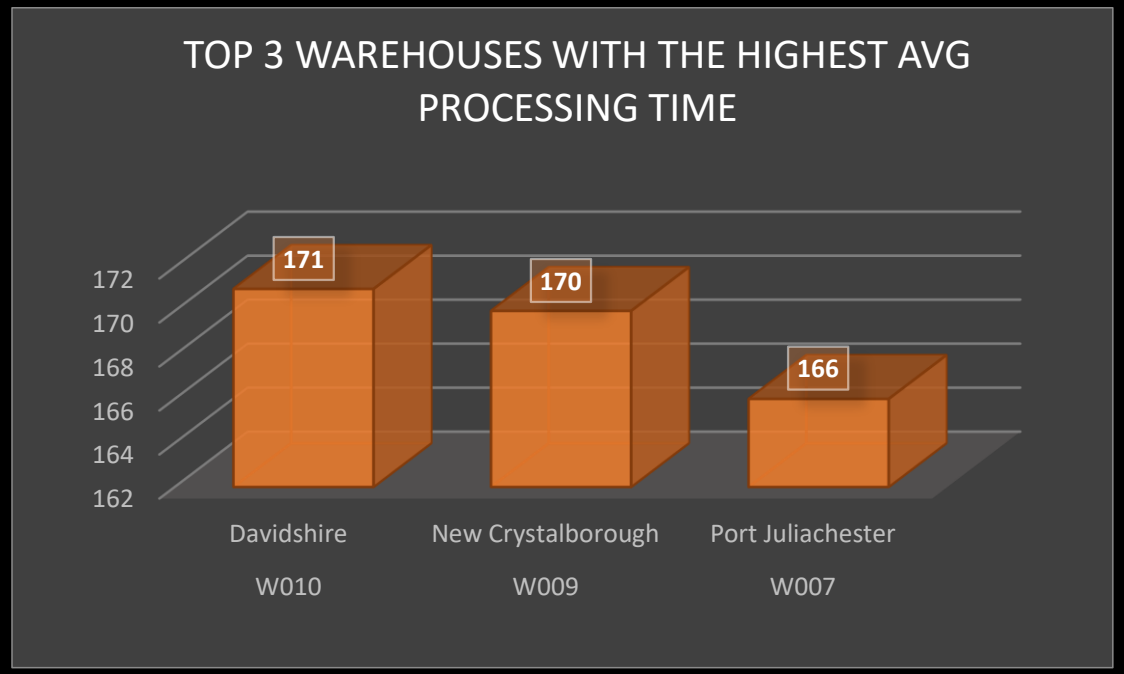
-- 2. Routes with high delay even within a short distance - If we comapare the fields, Average_Travel_Time against Distance_KM from the routes table we'll get to see many routes where the speed is significantly quite lower than the average. The issue here probably is frequent stops along the trip, UPS can use a routing strategy based on small geophraphic zones so that they don't have to carry out one exhausting long trip through a city with huge number of deliveries.

-- 3. Routes with hostile weather - If we cross-referrence the Delay_Reason field from the Shipment_Tracking table, we can see that Route_ID - R005 always shows "Weather" as Delay_Reason maybe because it uses a high-risk area (prone to hostile acts of nature, floods, etc.). UPS can pave an alternative path for a more flexible and practical rerouting strategy to resolve the matter.

4. Warehouse Performance

```
# The following query identifies and shows the top 3 warehouses with the
highest average processing time
SELECT
    Warehouse_ID,
    Location,
    ROUND(AVG(Processing_Time_Min)) AS avg_pro_time
FROM
    warehouses
GROUP BY Warehouse_ID , Location
ORDER BY avg_pro_time DESC
LIMIT 3;
```

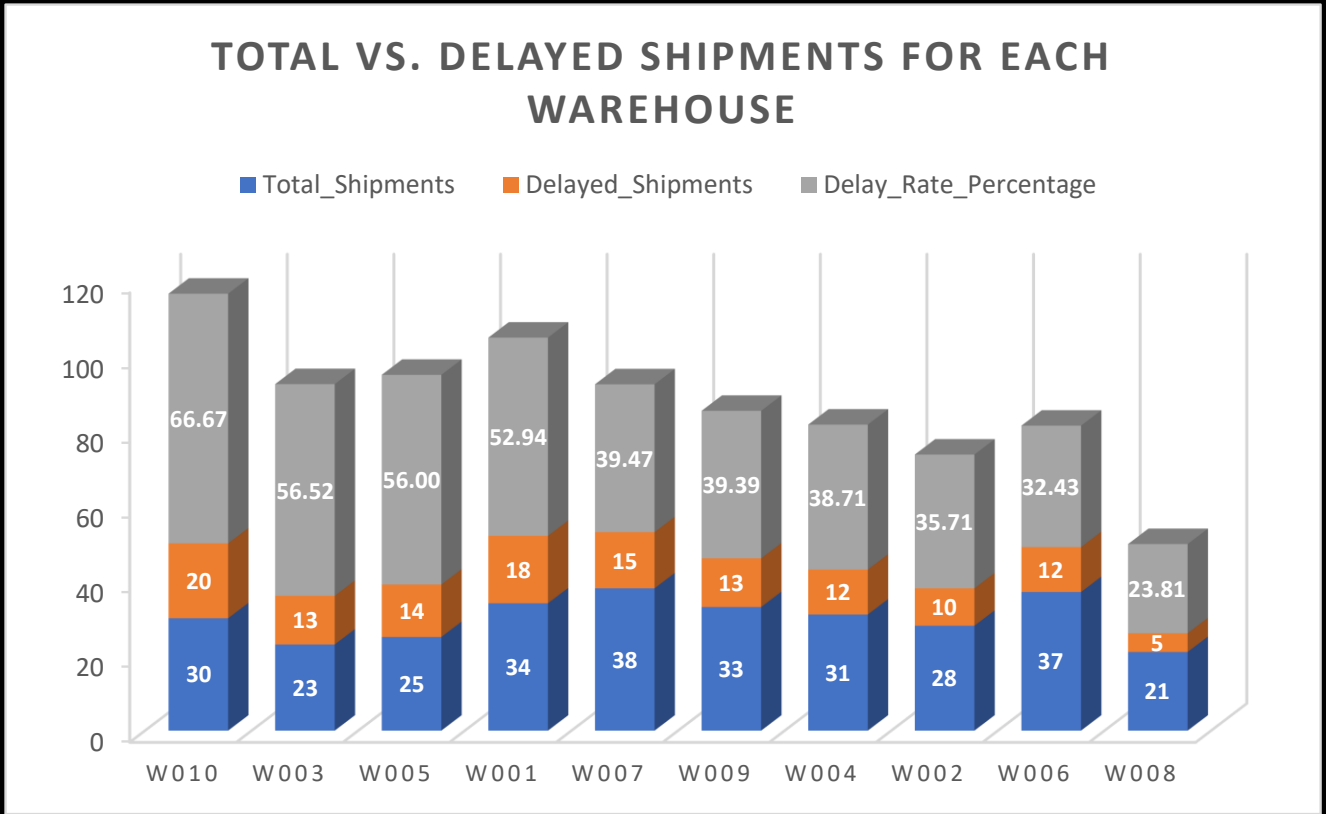
Warehouse_ID	Location	avg_pro_time
W010	Davidshire	171
W009	New Crystalborough	170
W007	Port Juliachester	166



```
# The query mentioned below compares total shipments with delayed shipments for each warehouse and
also calculates and shows delay rate for better analysis of the performance
SELECT
    Warehouse_ID,
    COUNT(Order_ID) AS Total_Shipments,
    SUM(CASE
        WHEN Delivery_Status = 'Delayed' THEN 1
        ELSE 0
    END) AS Delayed_Shipments,
    ROUND((SUM(CASE
                WHEN Delivery_Status = 'Delayed' THEN 1
                ELSE 0
            END) / COUNT(Order_ID)) * 100,
            2) AS Delay_Rate_Percentage
FROM
    Orders
GROUP BY Warehouse_ID
ORDER BY Delay_Rate_Percentage DESC;
```

Note : The result table of the query mentioned above is in the next slide.

Warehouse_ID	Total_Shipments	Delayed_Shipments	Delay_Rate_Percentage
W010	30	20	66.67
W003	23	13	56.52
W005	25	14	56.00
W001	34	18	52.94
W007	38	15	39.47
W009	33	13	39.39
W004	31	12	38.71
W002	28	10	35.71
W006	37	12	32.43
W008	21	5	23.81



```
# The following query is To identify bottleneck warehouses using CTEs to isolate the warehouse processing time and compare it against the global average processing time across all warehouses
WITH OrderProcessingTime AS (
  SELECT
      o.Order_ID,
      o.Warehouse_ID,
      DATEDIFF(
         STR_TO_DATE(MIN(t.Checkpoint_Time), '%d-%m-%Y'),
         STR_TO_DATE(o.Order_Date, '%d-%m-%Y')
      ) AS days_to_process
   FROM Orders o
   JOIN Shipment_Tracking t ON o.Order_ID = t.Order_ID
   WHERE t.Checkpoint = 'Checkpoint 1'
   GROUP BY o.Order_ID, o.Warehouse_ID, o.Order_Date
),
WarehousePerformance AS (
   SELECT
      Warehouse_ID,
      AVG(days_to_process) AS avg_warehouse_process_time
   FROM OrderProcessingTime
   GROUP BY Warehouse_ID
),
GlobalMetric AS (
   SELECT AVG(days_to_process) AS global_avg_time
   FROM OrderProcessingTime
)
SELECT
   wp.Warehouse_ID,
   ROUND(wp.avg_warehouse_process_time, 2) AS Avg_Processing_Days,
   ROUND(gm.global_avg_time, 2) AS Global_Avg_Days,
   ROUND(wp.avg_warehouse_process_time - gm.global_avg_time, 2) AS Days_Over_Average
FROM WarehousePerformance wp
CROSS JOIN GlobalMetric gm
WHERE wp.avg_warehouse_process_time > gm.global_avg_time
ORDER BY Days_Over_Average DESC;
```

When executed, the above query shows an empty table with only the field or column headers as required by the query because none of the records in the data met the conditions. The are no bottleneck warehouses with with higher processing time that the global average

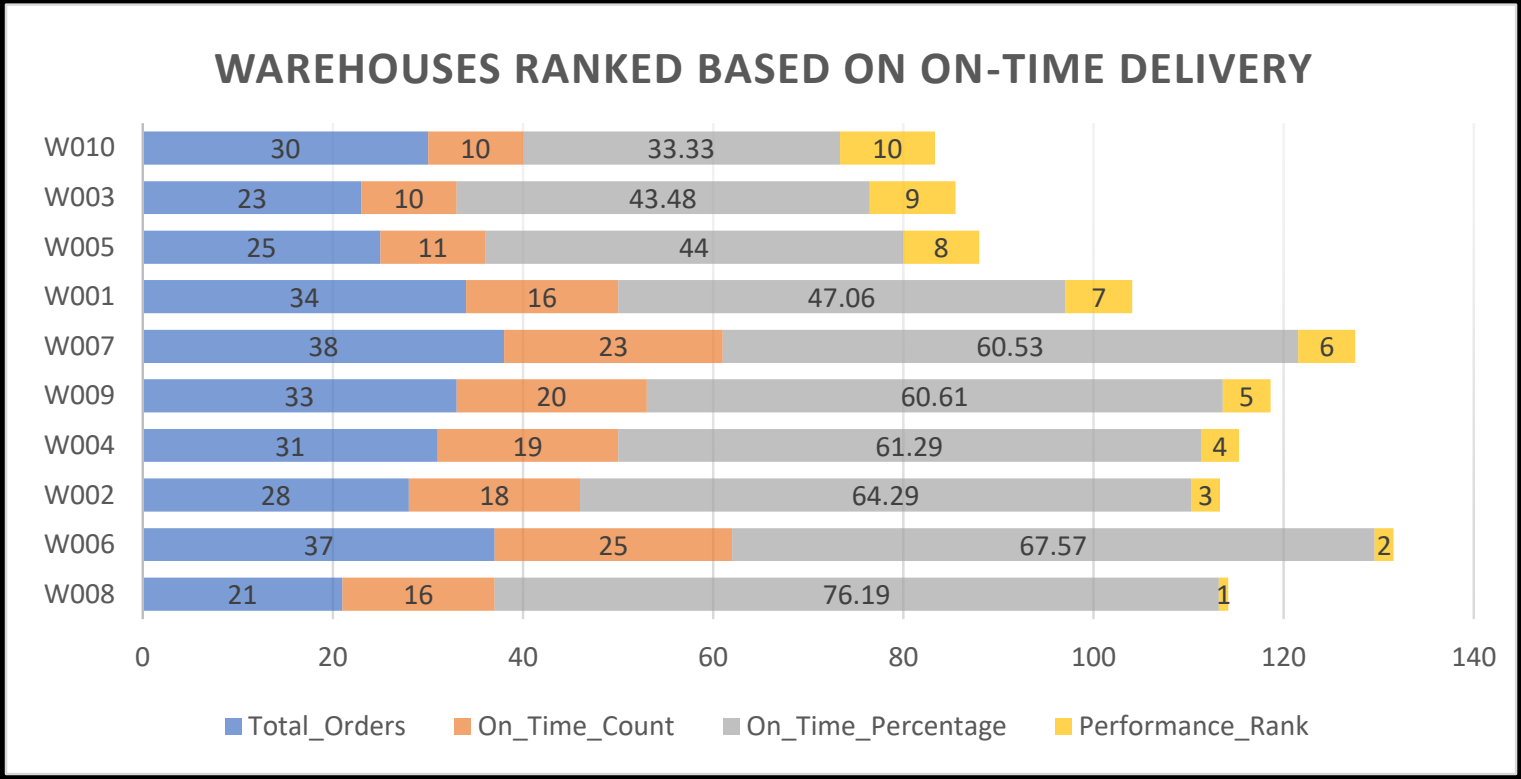
Warehouse_ID	Avg_Processing_Days	Global_Avg_Days	Days_Over_Average

The following query is used Rank to warehouses based on on-time delivery percentage for better Warehouse performance analysis

```
SELECT
   Warehouse_ID,
   COUNT(*) AS Total_Orders,
   SUM(CASE WHEN Delivery_Status = 'On Time' THEN 1 ELSE 0 END) AS On_Time_Count,
   ROUND(
      (SUM(CASE WHEN Delivery_Status = 'On Time' THEN 1 ELSE 0 END) / COUNT(*)) * 100,
      2
   ) AS On_Time_Percentage,
   RANK() OVER (
      ORDER BY (SUM(CASE WHEN Delivery_Status = 'On Time' THEN 1 ELSE 0 END) / COUNT(*)) DESC
   ) AS Performance_Rank
FROM Orders
GROUP BY Warehouse_ID
ORDER BY Performance_Rank;
```

Note : The result table of the last query mentioned above is in the next slide.

Warehouse_ID	Total_Orders	On_Time_Count	On_Time_Percentage	Performance_Rank
W008	21	16	76.19	1
W006	37	25	67.57	2
W002	28	18	64.29	3
W004	31	19	61.29	4
W009	33	20	60.61	5
W007	38	23	60.53	6
W001	34	16	47.06	7
W005	25	11	44	8
W003	23	10	43.48	9
W010	30	10	33.33	10



5. Delivery Agent Performance

```
# The following query is used to assign a rank to the agents per route by on-time delivery percentage
Select Agent_ID, Route_id, On_Time_Percentage, rank() over (Partition by Route_id order by On_Time_Percentage DESC) as agent_rank from deliveryagents;
```

Agent_ID	Route_id	On_Time_Percentage	agent_rank
A006	R001	81.56	1
A037	R001	78.94	2
A033	R002	98.01	1
A024	R002	97.16	2
A027	R002	73.80	3
A050	R002	72.75	4
A011	R002	71.50	5
A015	R003	83.31	1
A029	R005	81.71	1
A035	R005	74.50	2
A041	R006	91.47	1
A047	R006	84.62	2
A020	R006	77.65	3
A014	R007	99.90	1
A049	R007	99.82	2

Note : I have only used a preview of 15 rows in the result table above as there are 50 records here. After observing the result set in this case, it has come to my notice that Route ID – R002 has the highest overall on-time percentage and individually Agent ID – A014 passing through Route ID – R007 has the most on-time deliveries at 99.9%.

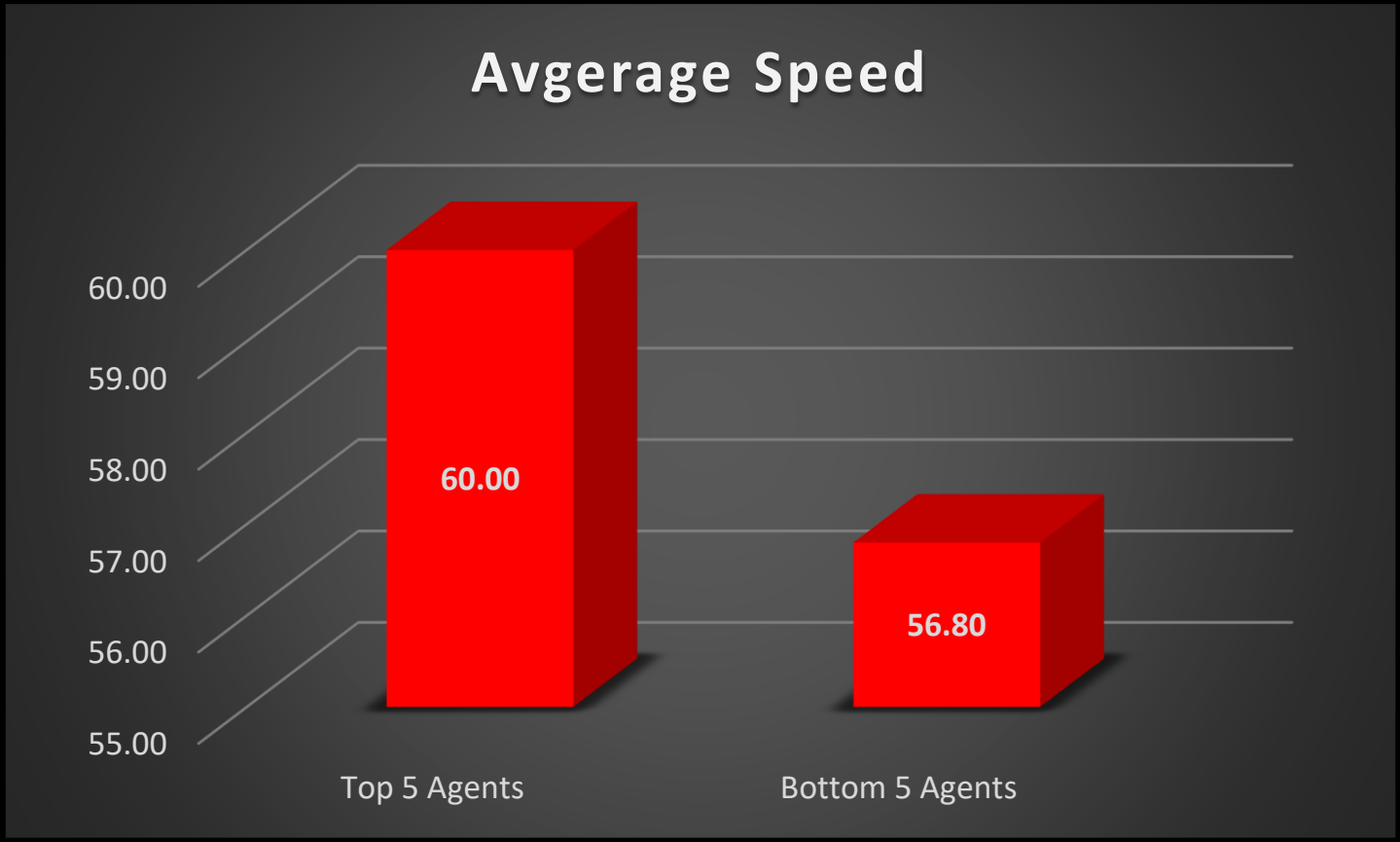
```
# The query below identifies and shows agents with deliveries on-time % < 80%.
SELECT
   Agent_ID, Route_ID, On_Time_Percentage
FROM
   DeliveryAgents
WHERE
   On_Time_Percentage < 80;
```

Agent_ID	Route_ID	On_Time_Percentage
A003	R014	74.15
A008	R018	72.39
A010	R019	70.45
A011	R002	71.50
A016	R012	78.39
A018	R007	73.21
A020	R006	77.65
A023	R017	74.21
A027	R002	73.80
A028	R007	74.47
A030	R014	76.60
A035	R005	74.50
A037	R001	78.94
A038	R018	71.80
A039	R016	72.60
A040	R007	77.83
A043	R016	79.51
A046	R020	75.46
A048	R012	77.92
A050	R002	72.75

```
# The query below compares the average speed of the top 5 vs the bottom 5 agents using subqueries
with UNION ALL
SELECT
    'Top 5 Agents' AS Category,
    ROUND(AVG(Avg_Speed_KM_HR), 2) AS Avg_Speed
FROM
    (SELECT
        Avg_Speed_KM_HR
    FROM
        DeliveryAgents
    ORDER BY On_Time_Percentage DESC
    LIMIT 5) AS top_agents
UNION ALL SELECT
    'Bottom 5 Agents' AS Category,
    ROUND(AVG(Avg_Speed_KM_HR), 2) AS Avg_Speed
FROM
    (SELECT
        Avg_Speed_KM_HR
    FROM
        DeliveryAgents
    ORDER BY On_Time_Percentage ASC
    LIMIT 5) AS bottom_agents;
```

Note : The result table of the last query mentioned above is in the next slide.

Category	Avg_Speed
Top 5 Agents	60.00
Bottom 5 Agents	56.80



6. Shipment Tracking Analytics

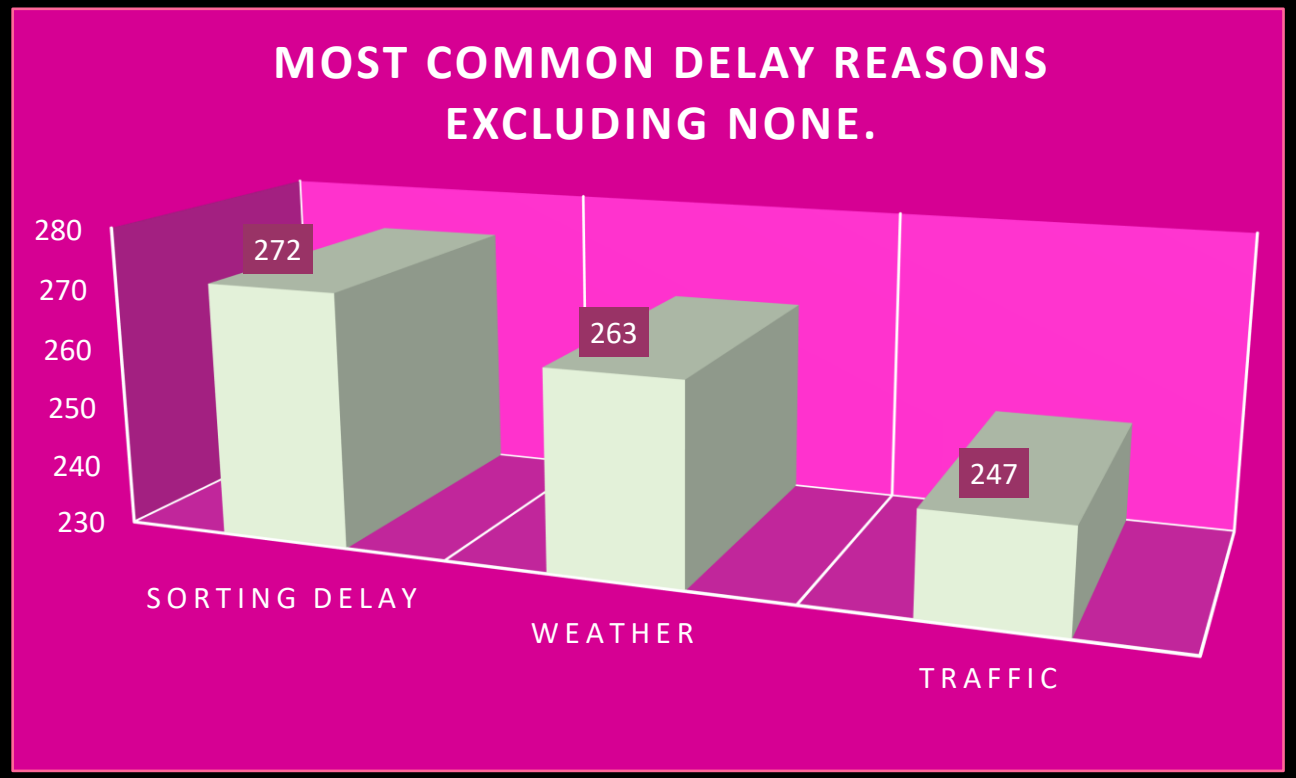
```
# The following query identifies and lists the last checkpoint and time for each
order using subqueries with the row_number() window function
Select Order_ID,
       Checkpoint,
       Checkpoint_Time
       from (
            Select Order_ID,
                   Checkpoint,
                   Checkpoint_Time,
                   row_number() over (
                                      Partition by Order_ID
                                      order by Checkpoint_Time
      desc
                                      ) as rn
              from shipment_tracking
            ) t
            where rn = 1;
```

Order_ID	Checkpoint	Checkpoint_Time
O0001	Checkpoint 1	09-06-2025
O0002	Checkpoint 3	23-06-2025
O0003	Checkpoint 2	05-06-2025
O0004	Checkpoint 2	29-06-2025

Note : I have only used a preview of 4 rows in the result table above as there are 300 records in this case.

```
# The query used below identifies and lists the most common delay reasons excluding
"None"
SELECT
    Delay_Reason, COUNT(*) AS total_occurrences
FROM
    shipment_tracking
WHERE
    Delay_Reason <> 'None'
GROUP BY Delay_Reason
ORDER BY total_occurrences DESC;
```

Delay_Reason	total_occurrences
Sorting Delay	272
Weather	263
Traffic	247



```
# The following query identifies orders with more than 2 delayed checkpoints
SELECT
    Order_ID, COUNT(*) AS delayed_chkpnts
FROM
    shipment_tracking
WHERE
    Delay_Reason <> 'None'
GROUP BY Order_ID
HAVING COUNT(*)> 2;
```

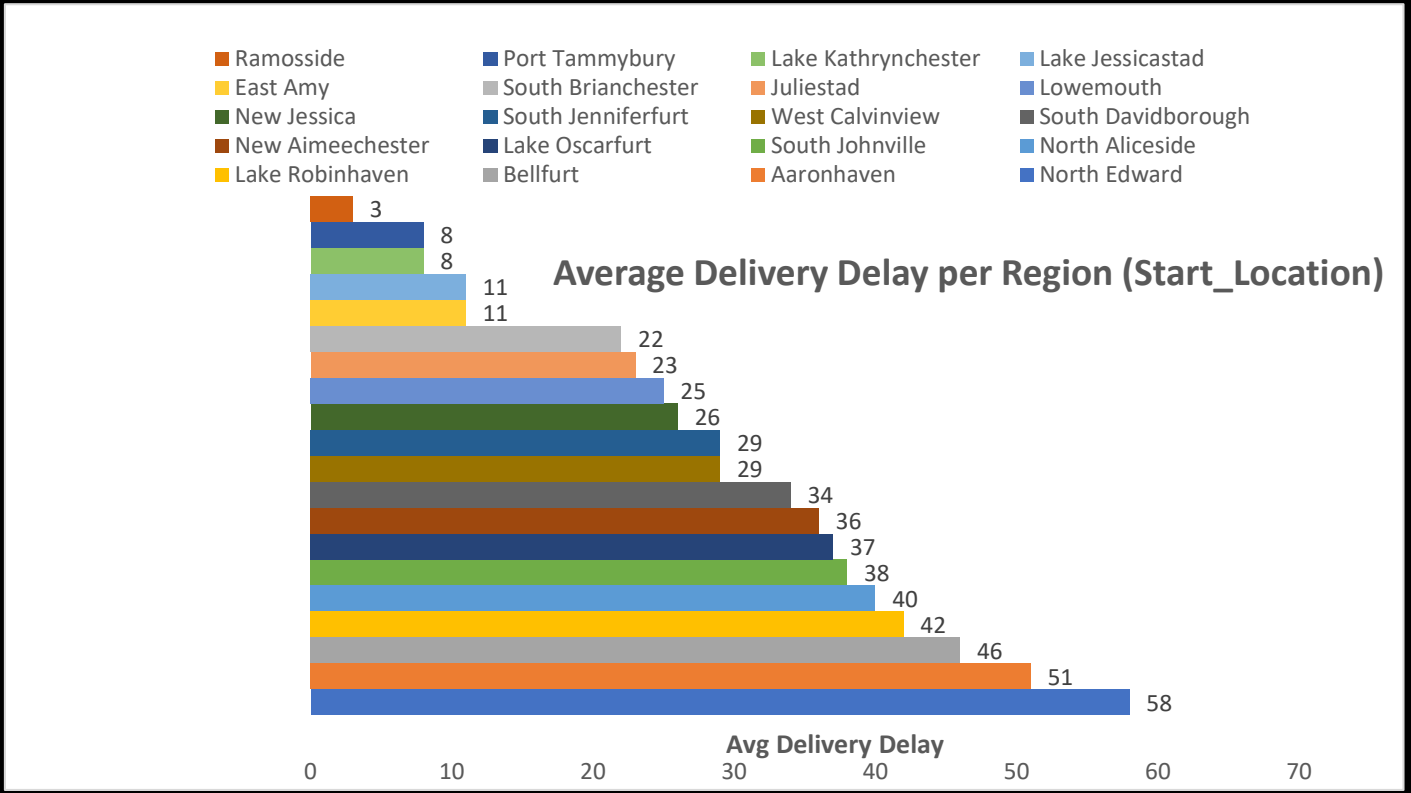
Order_ID	delayed_chkpnts
O0002	3
O0003	4
O0004	4
O0005	4
O0007	3
O0015	4
O0016	5
O0019	5
O0021	4
O0024	4

Note : I have only used a preview of 10 rows in the result table above as there are 151 records here. After observing the result set in it has become clear to me that the max number of delay checkpoints any order has is 5 and there are 13 total orders are on that list.

7. Advanced KPI Reporting

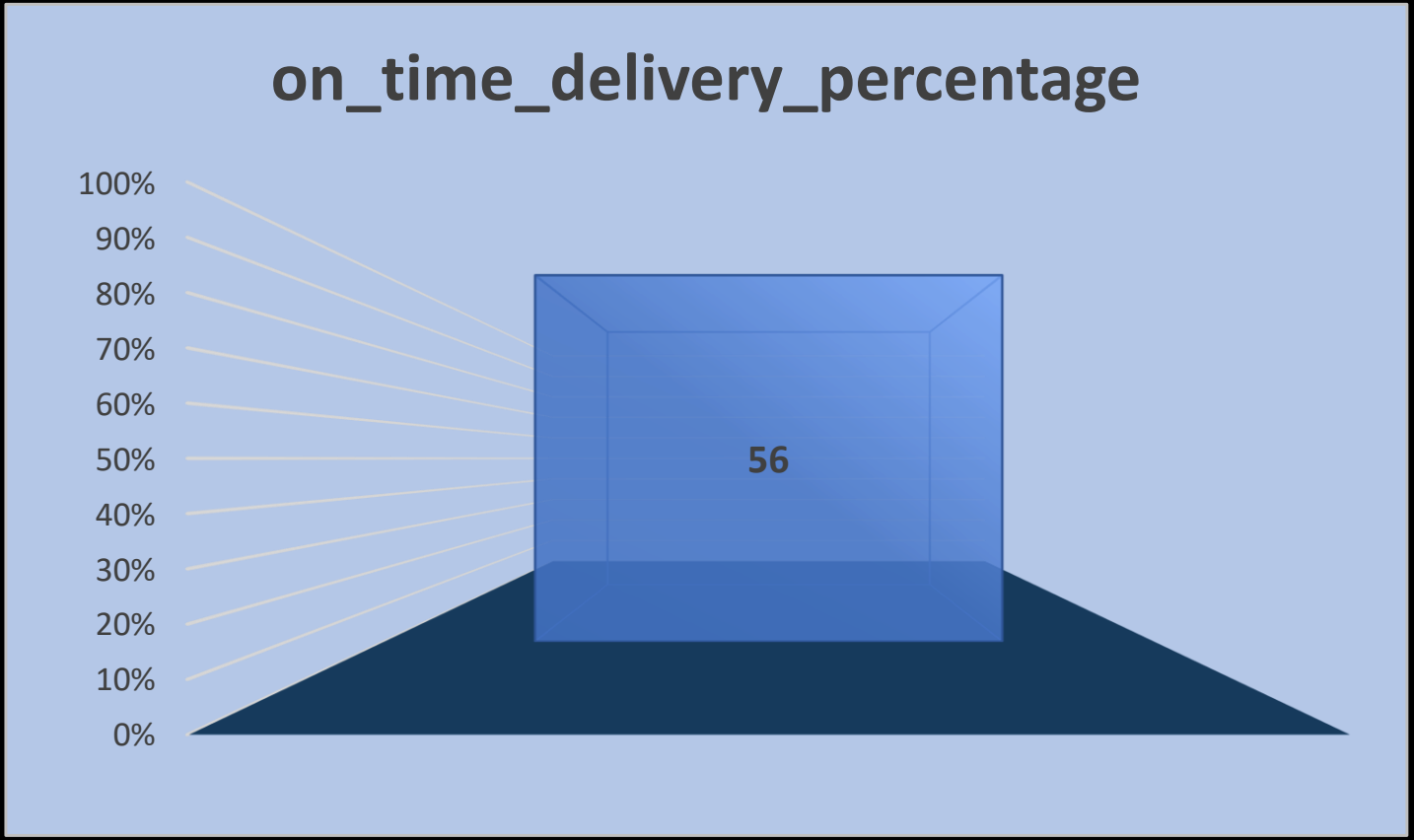
```
# The following query calculates and shows Average Delivery Delay per Region
(Start_Location)
SELECT
    Start_Location AS region,
    ROUND(AVG(Traffic_Delay_Min)) AS avg_del_delay
FROM
    routes
GROUP BY region
ORDER BY avg_del_delay DESC;
```

region	avg_del_delay
North Edward	58
Aaronhaven	51
Bellfurt	46
Lake Robinhaven	42
North Aliceside	40
South Johnville	38
Lake Oscarfurt	37
New Aimeechester	36
South Davidborough	34
West Calvinview	29
South Jenniferfurt	29
New Jessica	26
Lowemouth	25
Juliestad	23
South Brianchester	22
East Amy	11
Lake Jessicastad	11
Lake Kathrynchester	8
Port Tammybury	8
Ramosside	3



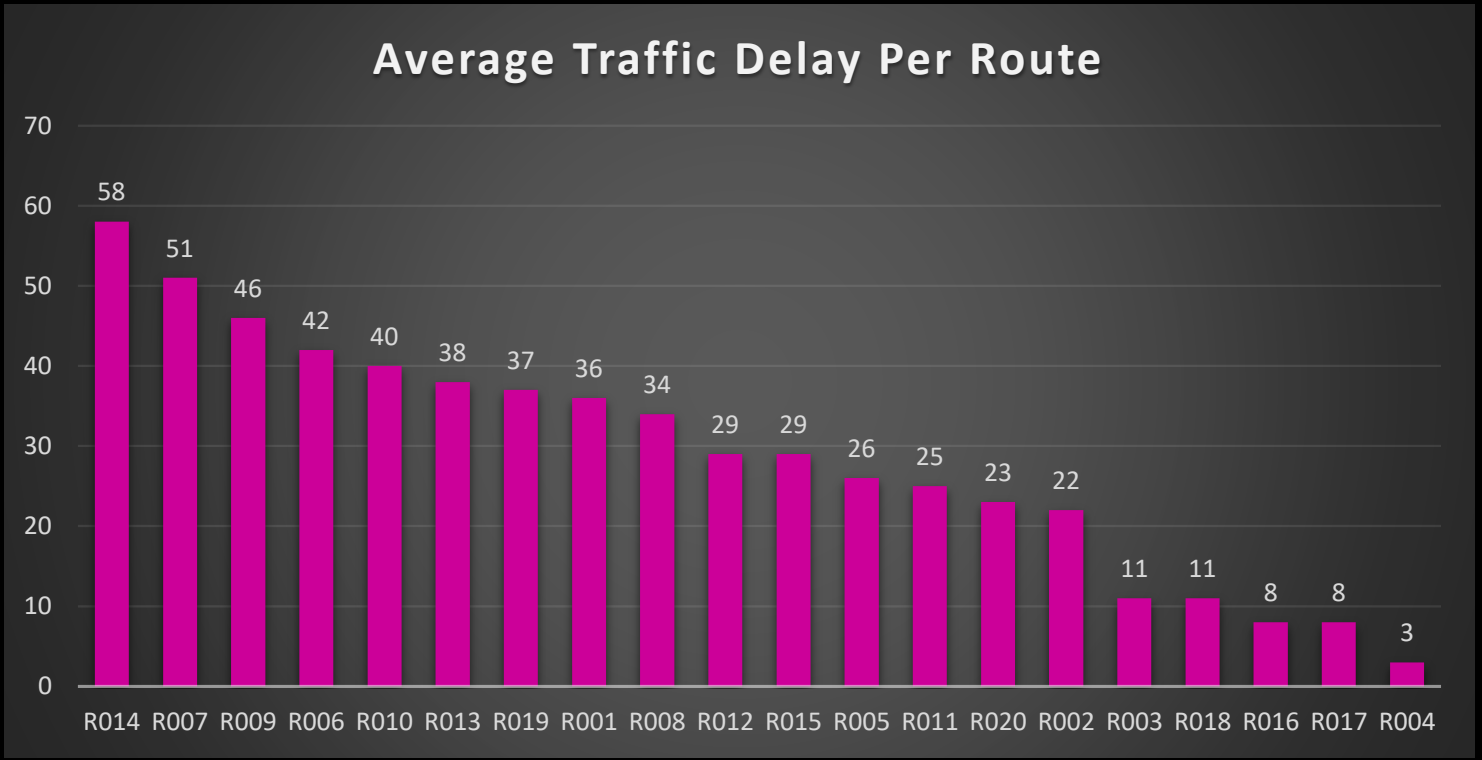
```
# The following query calculates On-Time Delivery % using the given formula (Total On-Time Deliveries / Total Deliveries) * 100
SELECT
    ROUND((SUM(CASE
                WHEN Delivery_Status = 'On Time' THEN 1
                ELSE 0
            END) / COUNT(*)) * 100
            ) AS on_time_delivery_percentage
FROM
    orders;
```

on_time_delivery_percentage
56



```
# The query below calculates and gives the average traffic delay for each route
SELECT
    Route_ID, AVG(Traffic_Delay_Min) AS avg_traffic_del
FROM
    routes
GROUP BY Route_ID
ORDER BY avg_traffic_del DESC;
```

Route_ID	avg_traffic_del
R014	58
R007	51
R009	46
R006	42
R010	40
R013	38
R019	37
R001	36
R008	34
R012	29
R015	29
R005	26
R011	25
R020	23
R002	22
R003	11
R018	11
R016	8
R017	8
R004	3



Drivelink for the Video Explaination :

https://drive.google.com/file/d/1PjIIrI4gh4jYrfmfjeTTbbmUibNPJzzz/view?usp=sharing